

Sommer 24

Del I

Konstanter

1. [5]

På hvilke måter (en eller flere) kan man deklarere en konstant i C++?

Skrive variabelnavnet med blokk-bokstaver.

Bruke nøkkelordet `const`.

Bruke nøkkelordet `const expr`.

Bruke nøkkelordet `const ant`.

Funksjoner

2. [5]

I hvilke (en eller flere) av situasjonene under burde man bruke pass-by-reference?

Når man gir en heltallsvariabel til en funksjon som regner ut kvadratroten og returnerer resultatet.

Når man gir et stort array av strenger til en funksjon som gjør alle forbokstaver store.

Når man gir en stor vektor av flyttall til en funksjon som regner ut gjennomsnittet av alle tallene i vektoren og returnerer gjennomsnittet.

Når man har en funksjon som tar inn en `unique_ptr`, men hvor funksjonen ikke endrer på verdien den peker til.

Klasser

3. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

En kopi-konstruktør kan brukes til å lage et nytt klasseobjekt ved å kopiere verdier fra et annet klasseobjekt av samme klasse.

Enhver klasse har en kopi-konstruktør.

En kopi-konstruktør kan defineres slik at den initialiserer et nytt klasseobjekt ved å kopiere verdier fra et annet klasseobjekt av en hvilken som helst klasse.

En kopi-konstruktør kan kun brukes til grunn kopiering (shallow copy).

enum-klasser

4. [5] Se på koden under.

```
1 enum class Color {  
2     red = 2, blue, yellow  
3 };  
4  
5 Color c = Color::yellow;
```

Hvilke (en eller flere) av følgende utsagn er korrekte?

Det er ikke mulig å definere en ny enum-klasse med verdiene `red`, `blue`, og `yellow`, fordi de allerede finnes i `Color`.

Verdiene til enumeratorene `red`, `blue` og `yellow` lagres som strenger.

Det er ikke mulig å direkte sammenligne en variabel av typen Color med en int.

Det er ikke mulig å tilordne verdien Color::yellow til en variabel fordi det er siste verdien som er definert i enum class.

Tilgangsnivå

5. [5]

Se på koden under.

```
class B {
    int y = 5;
protected:
    void f() const {}
    void g() {}
public:
    std::string s = "hello";

    virtual void c() = 0;
    void d() {}
};

class A : public B {
    int x;
public:
    void a() {}
    void b() {}
};
```

Hvor mange medlemsfunksjoner har klassen A? Du trenger ikke telle med funksjoner som er standard-definert.

- 5
- 2
- 4
- 6

RAII

6. [5]

Se på klassedeklarasjonen under.

```
class A {
    std::unique_ptr<std::ofstream> fileStream;

public:
    A(const string &filename) : fileStream(new std::ofstream(filename)) {}

    }
};
```

Hvilke (en eller flere) av følgende utsagn er korrekte?

Klassen overholder RAI (Resource Acquisition Is Initialization).

Klassen har en implisitt standard (default) destruktør.

Klassen mangler en destruktør som lukker filstrømmen `fileStream` for at den skal overholde RAII (Resource Acquisition Is Initialization).

Når en instans av klassen `Agår` ut av skop, oppstår det en minnelekkasje.

Beholdere

7. [5]

Hvilke (en eller flere) av de følgende beholderene har ikke elementene liggende rett etter hverandre i minnet?

`std::map`

`std::vector`

`std::set`

`std::array`

Lagring

8. [5]

Hvor lagres de ulike programelementene? (Kryss av i riktig boks.)

	Begge	Heap/ Free store	Stack
Medlemsvariabler			
Lokale variabler			
Funksjonsargumenter			
Et objekt allokert med <code>new</code> -operatoren			

`new`-operatoren

9. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

`new`-operatoren returnerer en peker til et objekt i minnet av en bestemt type.

`new`-operatoren allokerer alltid på stack-lageret (stack storage).

Man må bestemme hvor mange objekter av en type som skal allokeres før kjøretid når man bruker `new`-operatoren.

Uttrykket `int* a = new int {5};` vil allokere et array av 5 `int` på heap.

Templates

10. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

Kompilatoren kan i noen tilfeller løse template-parametre uten at typen til parametrene er oppgitt.

Klassetemplates kan ikke ha standard (default) template-parametre.

Template-argumenter kan kun være klasser.

Templates må plasseres i en header-fil. Templates må plasserast i ei header-fil.

I/O

11. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

Operativsystemet vil opprette en fil man prøver å skrive til hvis den ikke eksisterer.
En fil er en sekvens med bytes.
Operativsystemet vil opprette en fil man prøver å lese fra hvis den ikke eksisterer.
Det er ikke mulig å skrive private klassemedlemmer til fil.

Unntakshåndtering

12. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

Minne som har blitt allokert med new-operatoren i funksjonen blir automatisk frigjort når et unntak blir kastet.

Flere cat ch-blokker kan behandle samme type unntak.

Dersom det ikke finnes en cat ch-blokk for et utløst unntak, vil unntaket ignoreres og programmet fortsette normal kjøring fra punktet der unntaket ble utløst.

Programmet avslutter dersom et unntak ikke blir håndtert.

Del II

Bibliotek og linking

13. [5]

Forklar kort hva et bibliotek er og hvorfor linking er nyttig når man bruker bibliotek.

Funksjoner

14. [5] Forklar kort forskjellen på en funksjonsdeklarasjon og en funksjonsdefinisjon.

Typekonvertering

15. [5]

Forklar kort forskjellen på implisitt og eksplisitt typekonvertering.

Kopiering

16. [5]

Forklar kort forskjellen på grunn kopiering (shallow copy) og dyp kopiering (deep copy).

Beholdere og kopiering

17. [5]

Hvorfor er det unødvendig å bruke `std::unique_copy()` for å kopiere elementene i en `std::vector` til et `std::set`?

Beholdere og funksjoner

18. [5]

Forklar kort hva som skjer når man kaller `push_back()` på en `std::vector`.

Pekere

19. [5]

Se på de to kodesnuttene under.

Kodesnutt 1:

```
1 int * p = new int {10};
2 delete p;
3 delete p;
```

Kodesnutt 2:

```
1 int * p = new int {10};
2 p = new int {10};
3 delete p;
```

Kodesnutt 1 vil føre til en kjøretidsfeil, men det vil ikke kodesnutt 2. Forklar kort hvorfor.

Klasser

20. [5] Forklar kort hvilke problemer det er i kodesnutten under.

```
1 class Counter {
2 public:
3     void count(const int b) const {
4         a += b;
5         std::cout << "The counter is now set to: " << a << std::endl;
6     }
7
8 private:
9     int a = 0;
10 };
11
12 int main(void) {
13     int b = 1;
14     Counter a;
15     a.count(b);
16     return EXIT_SUCCESS;
17 }
```

21. [10] Se på koden under. Sjå på koden under.

```
1  #include <vector>
2
3  using namespace std;
4
5  class A {
6  private:
7      int x;
8  public:
9      A(int x_) : x(x_) {}
10 };
11
12 int main(void) {
13     unique_ptr<A> a {new A(5)};
14     vector<unique_ptr<A>> vec;
15     vec.push_back(a);
16
17     return 0;
18 }
```

(a) (5 points) Hvorfor kompilerer ikke koden? Forklar kort.

(b) (5 points) Hva ville du gjort for å løse problemet?

Templates

22. [10]

Se på koden under.

```
1  #include <iostream>
2
3  template<typename T>
4  struct Point {
5      T x = 0;
6      T y = 0;
7  };
8
9  int main(void) {
10     Point<int> p1{2, 4};
11     Point<int> p2{1, 6};
12
13     if (p1 > p2) {
14         std::cout << "The first point is largest" << std::endl;
```

```

15     } else if (p1 == p2) {
16         std::cout << "The points are equal" << std::endl;
17     } else {
18         std::cout << "The second point is largest" << std::endl;
19     }
20
21     return 0;
22 }

```

(a) (5 points) Hvorfor kompilerer ikke koden? Forklar kort.

(b) (5 points) Hva må gjøres for å løse problemet gitt at du ikke kan endre noe i `main`? Du trenger ikke skrive kode, men forklar kort hva du vil gjøre.